

SOFTWARE PROCESS SELECTION AND PROJECT PLAN

UNIVERSITY CORRIDOR MEETING ROOM AND WORKSPACE RESERVATION SYSTEM

TEAM: - SYED MEESUM RAZA NAQVI (24100183)

STUDENT ID	NAME
24100183	SYED MEESUM RAZA NAQVI

TABLE OF CONTENTS

1.	Introduction.....	3
2.	Software Process Selection.....	4
3.	Gantt Chart.....	7
4.	Risk Management	Error! Bookmark not defined.
4.1	Potential Risks and Mitigation Strategies	Error! Bookmark not defined.
5.	Development Environment Preparation.....	8
6.	Deployment Platform.....	9
7.	Who Did What?	10
8.	Review checklist	10

1. Introduction

<Give an overview of the project here. The overview must highlight the overall objectives of the project and its potential users and customers.>

Overview:

The University Corridor Meeting Room and Workspace Reservation System makes it easier to book meeting rooms and individual workspaces in different university departments. Each department has several corridors, and each corridor contains multiple rooms. Some of these rooms have several workspaces to meet different needs. Workspaces are of different types too. The individual workspace for like studying alone, shared workspace for group studying or office hours. Other rooms may be reserved exclusively for faculty meetings and do not have multiple workspaces; instead, the entire room will be booked for a specified time. The system helps manage these spaces better, reduces booking conflicts, and makes sure the rooms are used efficiently.

Objectives:

- To provide a centralized platform for booking meeting rooms and workspaces.
- To allow users to view, reserve, and manage bookings efficiently.
- To prevent double bookings and enforce booking policies to maintain fair usage.

Potential Users:

- Faculty members for departmental meetings.
- Teaching Assistants (TAs) for office hours and student consultations.
- Senior Year Students for studying alone or in group for project (restricted to 5 members).
- Application system for handling booking, conflict resolution, and notification.
- Admin for creating user accounts based on the user's status.

2. Software Process Selection

< (1) Discuss a pros and cons of waterfall and agile (scrum) processes in your own words.
(2) Select one of the above processes for your project development.
(3) Justify your selection with clear reasoning. Refer to “Project Context Analysis” in the slides to get help for writing this section.>

(1) Pros and Cons of Waterfall and Agile (Scrum) Processes

Waterfall Process:

- **Pros:**
 1. **Clear structure:** Waterfall follows a linear, sequential approach. Each phase must be completed before moving to the next, making it easy to understand and manage.
 2. **Well-defined requirements:** Waterfall works well when the requirements are stable and unlikely to change.
 3. **Emphasis on documentation:** Every stage produces detailed documentation, making it easier to track progress and changes.
 4. **Good for large teams:** It's suitable for larger teams that need to work on different phases simultaneously but sequentially.
- **Cons:**
 1. **Rigid and inflexible:** Once a phase is completed, it is hard to go back and make changes. This makes it unsuitable for projects where requirements may evolve.
 2. **Late delivery of the working system:** A working product is not seen until late in the project lifecycle, meaning issues may be discovered late in development.
 3. **Risk of misunderstanding requirements:** Since requirements are defined at the start and cannot change easily, any miscommunication can cause significant problems later.

Agile (Scrum) Process:

- **Pros:**
 1. **Flexibility:** Agile allows for iterative development, which is especially beneficial when requirements are expected to change frequently.

2. **Customer collaboration:** Frequent communication and feedback from the customer ensures that the development team is always aligned with the client's needs.
 3. **Early delivery of working software:** Agile focuses on delivering small, functional parts of the software early, allowing testing and feedback throughout the project lifecycle.
 4. **Adaptive:** Teams can easily adapt to changes, even late in development.
- **Cons:**
 1. **Less documentation:** Agile tends to prioritize working software over documentation, which can be an issue if detailed documentation is needed later.
 2. **Requires skilled team:** Agile relies heavily on team communication and collaboration. Without skilled team members, this could lead to mismanagement.
 3. **Potential for scope creep:** Frequent changes and feedback can lead to the project's scope expanding without proper control.

(2) Selected Process: Agile (Scrum)

For the development of the **University Corridor Meeting Room and Workspace Reservation System**, the **Agile (Scrum)** methodology is selected. The project needs flexibility, adaptability, and continuous feedback from users such as faculty members, teaching assistants (TAs), and senior students, which Agile is well-suited to handle.

(3) Justification for the Selection:

The **Agile (Scrum)** approach is selected for several reasons based on the project context:

1. **Changing Requirements:** The project involves a booking system, where user needs or system improvements may emerge during development. Agile's flexibility allows the team (single person in this case) to adapt to evolving requirements with minimal disruption.
2. **Continuous Feedback:** Agile enables frequent interaction with users (faculty, TAs, senior students). Feedback on early versions of the system (e.g., booking

module, conflict resolution) can be incorporated into subsequent sprints, ensuring the system meets user needs.

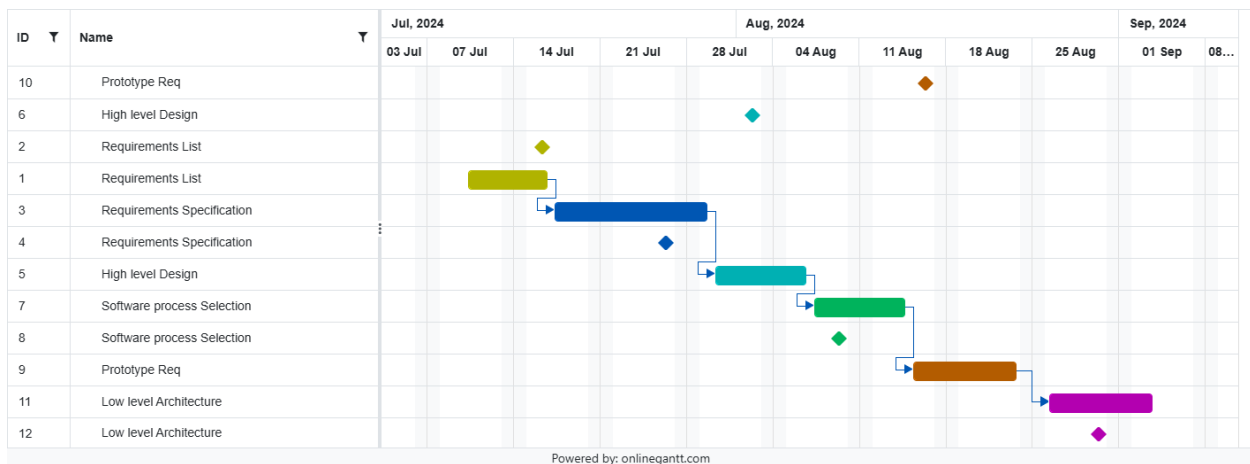
3. **Incremental Delivery:** The system will be developed in increments, with each sprint delivering a working feature. This early delivery ensures the system can be tested and used by users for feedback, reducing the risk of issues being discovered too late.
4. **Small Team Size:** Agile works well for small teams, allowing for better collaboration and flexibility. In this project, the small team will benefit from Agile's iterative process, delivering features progressively.
5. **Risk Mitigation:** Scrum's iterative nature allows risks to be identified and addressed early, particularly in features such as the conflict resolution module and notification system, which are critical to the success of the system.
6. **User-Centric Approach:** Agile's emphasis on user stories and backlog prioritization ensures that the most valuable features (e.g., booking conflicts, user notifications) are developed first, aligning the system closely with user expectations.

3. Gantt Chart

<Draw a Gantt chart that illustrates your project's schedule. The Gantt chart should show at least the following

- Tasks (tasks should not be too small or too large)
- Duration (in weeks)
- Milestones
- Team member names who are going to work on each task.

>



All tasks are to be done by Syed Meesum Raza.

4. Development Environment Preparation

- < (1) List down tools and technologies that you will use for prototype development.
(3) Setup the development environment on your machines and mention here that you have actually setup the environment. Include three snapshots of the tool(s) that you are going to use for development. These snapshots must be taken from the tool(s) while they are actually running on your system.>

4.1 Tools and Technologies

- **Front-End:**
 - **React.js** for building the user interface.
- **Back-End:**
 - **Node.js with Express.js** for handling the server-side logic.
 - **MongoDB** for database management.
- **Version Control:**
 - **GitHub** for version control and collaboration.
- **Testing:**
 - **JUnit** for unit testing.
 - **Selenium** for UI testing.
 - **Postman** for API testing.
- **Deployment:**
 - **Vercel** for deploying the front-end and back-end using serverless functions.

4.2 Development Environment Setup

- **MongoDB:** Making sure we have MongoDB set up locally or using MongoDB Atlas.
- **Node.js:** Ensuring that Node.js and Express are configured and connected to MongoDB.
- **React.js:** Ensuring the front-end is running locally with a React development server.
- **Git:** Using GitHub to track version history and manage code collaboration.

Screenshots:

- **React.js development server running on localhost.**
- **MongoDB running locally or through MongoDB Atlas.**

- **Node.js server running on localhost.**

5. Deployment Platform

<Find a free hosting service where you can deploy your prototype for anyone to access online>

The system will be deployed using **Vercel**, a serverless deployment platform that automatically scales with demand. Vercel allows for continuous integration and deployment through GitHub, meaning any updates made to the repository are automatically deployed to the live environment. The system's front-end (React) and back-end (Node.js) will be deployed on Vercel, and MongoDB Atlas will be used for the database.

Benefits:

- **Automatic scaling:** Vercel handles scaling during peak usage times.
- **Fast deployment:** Vercel allows rapid deployment with minimal configuration.
- **Serverless functions:** The back-end logic is handled via Vercel's serverless functions, reducing the complexity of managing servers.

6. Who Did What?

Name of the Team Member	Tasks done
Syed Meesum Raza Naqvi	Project planning, system architecture, front-end and back-end development, testing, deployment

7. Review checklist

Before submission of this deliverable, the team must perform an internal review. Each team member will review one or more sections of the deliverable.

Section Title	Reviewer Name(s)
Software Process Selection	Syed Meesum Raza Naqvi
Gantt Chart	Syed Meesum Raza Naqvi
Risk Management	Syed Meesum Raza Naqvi
Development Environment Setup	Syed Meesum Raza Naqvi
Deployment Platform	Syed Meesum Raza Naqvi